DESARROLLO DE APLICACIONES CON OBJETOS · UNIDAD 1

Prácticos de conjuntos, diccionarios y archivos

Los prácticos de clase con el enunciado tal cual, la solución en Python y la salida real con los archivos de la cátedra

4 prácticos | Resultados con los archivos reales | Código y datos para descargar

Cómo usar esta guía

Los cuatro prácticos de clase de conjuntos y diccionarios, con el **enunciado tal cual**, la solución y la **salida real** con los archivos de la cátedra (`numeros.txt`, `quijote.txt`, `words_alpha.txt` y `personas.csv`). Los archivos y el código están en el ZIP para correrlos en VS Code. Probado con Python 3.11.

	Práctico	Qué practica
1	**Números en archivo**	Leer un archivo y armar un conjunto (sin repetidos)
2	**Palabras del Quijote**	Diferencia de conjuntos con textos grandes
3	**Operaciones de conjuntos**	Unión, intersección y diferencia simétrica
4	**Padrón de personas**	Diccionario indexado por documento y un menú de búsquedas

1 Números en archivo

Enunciado

Del archivo numeros.txt que contiene un número entero por cada línea, generar un conjunto que contenga todos los números del archivo y desde el conjunto calcular e informar:

- Cantidad de números no repetidos
- Suma de todos los números
- Cantidad de impares
- Promedio de pares

Qué practica

Leer un archivo de texto línea por línea y armar un **conjunto por comprensión**: los repetidos quedan una sola vez.

Para tener en cuenta

- El archivo tiene 1000 líneas; en el conjunto quedan 622 números distintos. La suma, los impares y el promedio se calculan **desde el conjunto**, como pide el enunciado.

numeros.py

```python
def leer_numeros(nombre_archivo):
    """Un número entero por línea -> conjunto (los repetidos quedan una sola vez)."""
    with open(nombre_archivo, encoding="utf-8") as archivo:
        return {int(linea) for linea in archivo if linea.strip()}


def main():
    numeros = leer_numeros("numeros.txt")
    pares = [n for n in numeros if n % 2 == 0]
    print("Cantidad de números no repetidos:", len(numeros))
    print("Suma de todos los números:", sum(numeros))
    print("Cantidad de impares:", len([n for n in numeros if n % 2 != 0]))
    print(f"Promedio de los pares: {sum(pares) / len(pares):.2f}" if pares else "No hay pares")


if __name__ == "__main__":
    main()
```

Salida al correrlo

```
Cantidad de números no repetidos: 622
Suma de todos los números: 314220
Cantidad de impares: 318
Promedio de los pares: 507.45
```

2 Palabras del Quijote

Enunciado

Procesamiento de palabras del Quijote. Tomando el texto del libro “El Quijote de La Mancha” y el diccionario del idioma inglés calcular e informar:

- Cantidad de palabras únicas (sin repetición) del libro.
- Cantidad de palabras del diccionario.
- Cantidad de palabras del libro que no existen en el diccionario.
- Listado ordenados de todas las palabras que no existen.

Qué practica

Conjuntos con textos grandes: la **diferencia** `libro - diccionario` responde de una sola vez qué palabras no existen.

Para tener en cuenta

- El `quijote.txt` de la cátedra es la traducción **al inglés** (la de John Ormsby, del Proyecto Gutenberg): por eso se compara con un diccionario de inglés.
- Se pasa todo a minúscula y se toman como palabras las secuencias de letras (`re.findall`), así “Sancho,” y “sancho” cuentan como la misma.
- El listado completo tiene 1211 palabras: acá se muestran las primeras y las últimas.

quijote.py

```python
import re


def palabras_de(nombre_archivo):
    """Todas las palabras del texto, en minúscula y sin signos de puntuación."""
    with open(nombre_archivo, encoding="utf-8") as archivo:
        return re.findall(r"[a-záéíóúüñ]+", archivo.read().lower())


def leer_diccionario(nombre_archivo):
    with open(nombre_archivo, encoding="utf-8") as archivo:
        return {linea.strip() for linea in archivo if linea.strip()}


def main():
    libro = set(palabras_de("quijote.txt"))          # el texto es la traducción al inglés
    diccionario = leer_diccionario("words_alpha.txt")
    no_existen = libro - diccionario                  # diferencia de conjuntos

    print("Palabras únicas del libro:", len(libro))
    print("Palabras del diccionario:", len(diccionario))
    print("Palabras del libro que no están en el diccionario:", len(no_existen))
    print("Listado ordenado:")
    for palabra in sorted(no_existen):
        print("  ", palabra)


if __name__ == "__main__":
    main()
```

Salida al correrlo

```
Palabras únicas del libro: 15716
Palabras del diccionario: 370105
Palabras del libro que no están en el diccionario: 1211
Listado ordenado:
   abajo
   abernuncio
   abideth
   abindarraez
   abrenuncio
   achilleses
   acquaviva
   adriani
   adversae
   aequo
   agilers
   agrajes
   agramante
   aguero
   aguilar
   ainting
   alagones
   alastrajareas
   albogue
   albogues
   albraca
   albracca
… (1183 renglones más)
   zonzorino
   zopyrus
   zoraida
   zoroastric
   zorruna
   zulema
```

3 Operaciones de conjuntos

Enunciado

Palabras Únicas: Toma una cadena como entrada y crea un conjunto que contenga las palabras únicas presentes en esa cadena.

Palabras Comunes: Dados dos textos, encuentra las palabras que son comunes en ambos textos utilizando conjuntos.

Elementos Únicos en Listas: Toma dos listas y crea un conjunto que contenga los elementos que son únicos en ambas listas (no presentes en la otra lista).

Qué practica

Las operaciones entre conjuntos: `&` intersección, `|` unión, `-` diferencia y `^` diferencia simétrica (lo que está en uno solo de los dos).

Para tener en cuenta

- Un conjunto no tiene orden: al imprimirlo, los elementos pueden salir en otro orden.

conjuntos.py

```python
def palabras_unicas(cadena):
    """Conjunto con las palabras de la cadena, sin repetir."""
    return set(cadena.lower().split())


def palabras_comunes(texto1, texto2):
    """Palabras que aparecen en los dos textos (intersección)."""
    return palabras_unicas(texto1) & palabras_unicas(texto2)


def elementos_unicos(lista1, lista2):
    """Elementos que están en una sola de las dos listas (diferencia simétrica)."""
    return set(lista1) ^ set(lista2)


def main():
    print(palabras_unicas("el perro y el gato y el loro"))
    print(palabras_comunes("me gusta programar en Python", "en Python todo es un objeto"))
    print(elementos_unicos([1, 2, 3, 4, 5], [4, 5, 6, 7]))


if __name__ == "__main__":
    main()
```

Salida al correrlo

```
{'gato', 'perro', 'y', 'loro', 'el'}
{'python', 'en'}
{1, 2, 3, 6, 7}
```

4 Padrón de personas

Enunciado

El archivo personas.csv contiene un padrón de personas a razón de una persona por línea y en cada una separadas con comas el documento, nombre, apellido y edad. Desarrollar un programa en python que lea el archivo y guarde todo su contenido en un diccionario indexado por documento. Luego el programa debe ofrecer un menú con las siguientes opciones:

- Búsqueda por documento: que solicite un documento y si lo encuentra muestre todos los datos de la persona encontrada y un mensaje adecuado si no la encuentra.
- Búsqueda por apellido: que solicite un apellido y muestre por pantalla todos los datos de todas las personas cuyo apellido sea igual al ingresado.
- Mostrar el promedio de edades de todos.

Qué practica

Un **diccionario indexado**: buscar por documento es directo (`documento in padron`), mientras que por apellido hay que recorrerlo.

Para tener en cuenta

- El apellido se pasa a mayúscula antes de buscar, porque el archivo está en mayúsculas.

padron.py

```python
def leer_padron(nombre_archivo):
    """documento,nombre,apellido,edad -> diccionario indexado por documento."""
    padron = {}
    with open(nombre_archivo, encoding="utf-8") as archivo:
        for linea in archivo:
            if linea.strip():
                documento, nombre, apellido, edad = linea.strip().split(",")
                padron[documento] = {"nombre": nombre, "apellido": apellido, "edad": int(edad)}
    return padron


def mostrar(documento, persona):
    print(f"   {documento}: {persona['apellido']}, {persona['nombre']} ({persona['edad']} años)")


def main():
    padron = leer_padron("personas.csv")
    opcion = ""
    while opcion != "0":
        print("1. Buscar por documento  2. Buscar por apellido  3. Promedio de edades  0. Salir")
        opcion = input("Opción: ")
        if opcion == "1":
            documento = input("Documento: ")
            if documento in padron:                        # búsqueda directa por clave
                mostrar(documento, padron[documento])
            else:
                print("   No hay ninguna persona con ese documento")
        elif opcion == "2":
            apellido = input("Apellido: ").upper()
            encontrados = {d: p for d, p in padron.items() if p["apellido"] == apellido}
            for documento, persona in encontrados.items():
                mostrar(documento, persona)
            print(f"   {len(encontrados)} persona(s) con apellido {apellido}")
        elif opcion == "3":
            promedio = sum(p["edad"] for p in padron.values()) / len(padron)
            print(f"   Promedio de edades: {promedio:.2f}")


if __name__ == "__main__":
    main()
```

Salida al correrlo

```
1. Buscar por documento  2. Buscar por apellido  3. Promedio de edades  0. Salir
Opción: 1
Documento: 22861213
   22861213: GIL, RUBEN (14 años)
1. Buscar por documento  2. Buscar por apellido  3. Promedio de edades  0. Salir
Opción: 1
Documento: 11111111
   No hay ninguna persona con ese documento
1. Buscar por documento  2. Buscar por apellido  3. Promedio de edades  0. Salir
Opción: 2
Apellido: gil
   22861213: GIL, RUBEN (14 años)
   90775752: GIL, AMPARO (51 años)
   88967431: GIL, ELENA (69 años)
   17412261: GIL, MONICA (95 años)
   61397287: GIL, ALBERTO (58 años)
   66738409: GIL, HUGO (25 años)
   18119690: GIL, NOELIA (61 años)
   47989931: GIL, LORENA (41 años)
   46414251: GIL, ANDREA (94 años)
   81756892: GIL, GONZALO (11 años)
   43567913: GIL, GLORIA (32 años)
   74468513: GIL, JOSE CARLOS (32 años)
   85872350: GIL, LAURA (81 años)
   66992702: GIL, ALICIA (90 años)
   49527716: GIL, ROSA (57 años)
   76956793: GIL, FRANCISCO JAVIER (95 años)
   54746411: GIL, JAVIER (82 años)
   50135659: GIL, NATALIA (36 años)
   14931918: GIL, ENCARNACION (77 años)
   49997294: GIL, FRANCISCO JOSE (18 años)
   45593204: GIL, MARIA ROSA (52 años)
   13538902: GIL, CLAUDIA (31 años)
   21801477: GIL, RAUL (73 años)
   83494736: GIL, LORENZO (68 años)
   24 persona(s) con apellido GIL
1. Buscar por documento  2. Buscar por apellido  3. Promedio de edades  0. Salir
Opción: 3
   Promedio de edades: 54.88
1. Buscar por documento  2. Buscar por apellido  3. Promedio de edades  0. Salir
Opción: 0
```